PlagiarismGuard

A Full-Stack Word-Level Plagiarism Checker

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by

Duddukuri Sri Tanmai (2520080042)

Akshitha V (2520080003)

Team Number: [Team Number]

Section: 12

Under the guidance of

Chandusha K

Asst. Professor, CSIT Department



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



DECLARATION

The Mini Project Report entitled "PlagiarismGuard: A Full-Stack Word-Level Plagiarism Checker" is a record of Bonafide work of Duddukuri Sri Tanmai – 2520080042, Akshitha V – 2520080003 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Duddukuri Sri Tanmai (2520080042)
Akshitha V (2520080003)

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled "PlagiarismGuard: A Full-Stack Word-Level Plagiarism Checker" is a bonafide work done and submitted by Duddukuri Sri Tanmai – 2520080042, Akshitha V – 2520080003 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year 2026-2027.

Signature of the Supervisor

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance of many people. We wish to express our sincere thanks to all those who assisted us in the completion of this project.

Our greatest appreciation goes to our guide, Chandusha K, Asst. Professor, Department of Computer Science and Information Technology, for the tremendous support, encouragement, and guidance provided throughout this project.

We express our gratitude to Dr. P. Venkateshwar Rao, Head of the Department of Computer Science Engineering, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends, and classmates for their kind help and cooperation during our work.

Duddukuri Sri Tanmai (2520080042)
Akshitha V (2520080003)

TABLE OF CONTENTS

S.No.	Contents	Page no.
1	Abstract	6
2	Introduction	6
3	Literature survey	7
4	Problem Statement and Objectives	8
5	Hardware and Software Requirements	9
6	Implementation	10
7	Results & Discussion	12
8	Conclusion & Future Scope	13
9	References	13

1. Abstract

Academic and professional writing is frequently checked for plagiarism, but most available checkers are either closed-source, cloud-dependent commercial products, or simple exact-match tools that miss reworded or reordered copying. This project, PlagiarismGuard, is a full-stack, word-level plagiarism checker built to address both gaps: it detects similarity using hand-implemented classical algorithms — a Rabin–Karp rolling-hash matcher for verbatim copying and a Longest Common Subsequence (LCS) dynamic-programming aligner for paraphrased or reordered text — without relying on any third-party plagiarism or diff library. The system is split into two independently runnable halves: a Java 21 backend that performs tokenisation, matching, and scoring, and exposes its results over a small JDK-only HTTP/JSON API (built with com.sun.net.httpserver, requiring no external framework); and a lightweight HTML/CSS/JavaScript frontend that collects a submission and one or more reference documents, calls that API, and renders a similarity score, a per-source breakdown, and a highlighted report distinguishing exact matches from paraphrased spans. The two halves were originally developed and uploaded separately and were connected during this project by adding a thin API layer (Server.java and a hand-written JSON parser, Json.java) on top of the existing, unmodified checker classes. The result is a working, demonstrable application that reinforces core Data Structures and Algorithms concepts — hashing, string processing, and dynamic programming — through a realistic client-server system.

2. Introduction

Plagiarism, whether intentional or accidental, undermines the integrity of academic and professional written work. Simple copy-paste detection is not sufficient on its own, since even minor edits — reordering sentences, swapping synonyms, or rephrasing — can defeat naive string-comparison tools. At the same time, full-scale commercial systems such as Turnitin are costly, require constant internet connectivity to a vendor's servers, and give no visibility into how their similarity scores are actually computed, which limits their usefulness both as a learning exercise and as a tool a small team can run and extend on its own.

PlagiarismGuard addresses this gap with a transparent, self-hosted checker built from first principles. The backend's matching logic (Tokenizer, RabinKarpMatcher, LCSAligner, SimilarityScorer) was written and compiled independently of any web framework, and was originally runnable only as a command-line tool. The frontend was built separately as a static web page with its own, simplified, in-browser approximation of the same algorithms. The core engineering contribution of this phase of the project was to connect the two: a small HTTP server (Server.java) was added that wraps the existing backend classes exactly as they were — no

algorithm code was changed — and exposes a POST /api/check endpoint; the frontend's JavaScript was then rewritten to call that endpoint over fetch() instead of running its own matching logic, so that what the user sees on screen is now the output of the real, hand-implemented Java algorithms rather than a browser-side approximation.

Objectives

1. Preprocess documents into normalised token streams suitable for comparison (Tokenizer, Document, Token).
2. Detect verbatim copying efficiently using a rolling-hash (Rabin–Karp) technique (RabinKarpMatcher).
3. Detect paraphrased or reordered copying using dynamic-programming LCS alignment on the tokens left unmatched by the exact-match stage (LCSAligner).
4. Combine both signals into a single similarity score, a per-source percentage breakdown, and a list of matched spans (SimilarityScorer, ScoreResult).
5. Expose the checker as a small, dependency-free HTTP/JSON API so it can be called from a browser (Server.java, Json.java).
6. Connect an existing browser-based frontend to that API and render a highlighted, human-readable report (ReportGenerator on the backend; app.js on the frontend).
7. Evaluate the system against hand-labelled original, copied, and paraphrased documents (Evaluator, SanityTest).

3. Literature Survey

A review of prior work in string matching, similarity detection, and existing plagiarism-checking systems informed the algorithmic and architectural choices made in this project.

S.No	Project / source	Key contribution	Relation to this project
1	Karp & Rabin (1987) — Efficient Randomized Pattern-Matching Algorithms	Introduced rolling-hash based string matching, enabling fast average-case exact substring search	Basis for the RabinKarpMatcher exact-match stage
2	Hunt & McIlroy (1976) — An Algorithm for Differential File Comparison	Formalised the Longest Common Subsequence (LCS) approach used in diff tools	Basis for the LCSAligner paraphrase-detection stage
3	MOSS — Measure of Software Similarity (Schleimer et al., 2003)	Introduced winnowing-based fingerprinting to cheaply shortlist similar documents before detailed comparison	Informed the two-stage (cheap filter, then precise alignment) design philosophy

4	Turnitin	Commercial, cloud-based plagiarism detection with document repositories and similarity reports	Reference point for report format (score, source breakdown, highlighted matches); this project is open and self-hosted by contrast
5	Grammarly Plagiarism Checker	Browser-based checker matching text against indexed web content	Reference for UI expectations (score display, flagged passages)
6	Copyleaks	API-driven plagiarism detection service with a REST interface	Reference for the client-server split used in this project's own API
7	Quetext	Web-based checker using DeepSearch technology for contextual matching	Reference for paraphrase/contextual detection as a feature beyond exact match
8	JPlag	Academic tool for detecting similarity in programming assignments using tokenisation and string matching	Reference for building a checker from first principles rather than wrapping a third-party library
9	PlagScan	Educational plagiarism-detection SaaS with per-institution document databases	Reference for the reference-corpus model (submission checked against a set of uploaded sources)
10	Google's MinHash/LSH literature (used in near-duplicate web page detection)	Locality-sensitive hashing for approximate similarity at scale	Discussed as a scalability extension in Future Scope

The survey highlights a two-stage design that recurs across both the academic literature and the commercial tools reviewed: a cheap technique (hashing or fingerprinting) is used to cheaply find or shortlist exact or near-exact matches, while a more expensive but more precise technique (sequence alignment) is reserved for catching reworded or reordered copying that the cheap stage misses. PlagiarismGuard adopts exactly this pattern — Rabin–Karp first, LCS second — and additionally mirrors the client-server split used by API-driven commercial tools such as Copyleaks, which was the direct motivation for adding an HTTP layer on top of the existing command-line checker rather than keeping it as a CLI-only tool.

4. Problem Statement and Objectives

Problem Statement

Existing plagiarism checkers are either too simplistic to catch reworded copying, or too opaque and costly to use as a learning tool or a self-hosted solution.

Problem Description

A checker that only compares exact strings misses paraphrased plagiarism, while commercial cloud-based tools hide their methodology and require ongoing internet access and licensing. There is a need for a transparent, locally runnable checker that combines both exact and paraphrase detection and can be operated as a simple two-process web application.

Objectives

The objective of this project was twofold: first, to build a correct, hand-implemented plagiarism-detection engine covering both exact and paraphrase detection using classical DSA techniques; and second, to make that engine usable as a real application by connecting it to a web frontend through a minimal, dependency-free HTTP API — turning two separately built artifacts (a CLI tool and a static web page) into one working, demonstrable system.

5. Hardware and Software Requirements

Hardware Requirements

Component	Minimum specification
Processor	Dual-core 2.0 GHz or higher (Intel i3 / AMD equivalent)
RAM	4 GB minimum (8 GB recommended for larger reference corpora)
Storage	500 MB free space for source code, class files, and test documents
Display	1366x768 or higher resolution
Network	Not required at runtime — both servers run on localhost; only needed for setup/version control

Software Requirements

Category	Details
Operating System	Windows 10/11, Linux (Ubuntu 20.04+), or macOS

Backend language/runtime	Java 21 (JDK) — javac, java, and the built-in jwebserver utility
Backend libraries	None — com.sun.net.httpserver.HttpServer (JDK-bundled) and a hand-written JSON parser/writer (Json.java)
Frontend	HTML5, CSS3, vanilla JavaScript (ES6+) — no build step, no framework
IDE	VS Code / IntelliJ IDEA / Eclipse
Version control	Git and GitHub for source management
Testing	SanityTest.java (custom) for backend algorithm checks; manual browser testing for the frontend

6. Implementation

The system is organised into six modules spanning two layers — the Java backend, which performs all matching and scoring, and the browser frontend, which collects input and renders results. The backend modules were implemented and validated independently (via SanityTest and the CLI entry point, Main.java) before the API bridge was added to connect them to the frontend.

Module	Responsibility	Key class(es)	Layer
Preprocessing	Reads raw text, tokenises into words, normalises case/punctuation, optionally strips stop words	Tokenizer, Token, Document	Backend
Exact-match detection	Rolling-hash (Rabin–Karp) comparison between submission and each reference to find verbatim-copied spans	RabinKarpMatcher, MatchSpan	Backend
Paraphrase detection	Dynamic-programming LCS alignment on the remaining unmatched tokens to catch reworded/reordered copying	LCSAligner	Backend
Scoring & reporting	Combines exact and paraphrase spans into a similarity percentage, a	SimilarityScorer, ScoreResult, ReportGenerator	Backend

	per-source breakdown, and a highlighted report		
API bridge	Exposes the checker over HTTP as JSON (POST /api/check, GET /api/health); has no algorithmic logic of its own	Server, Json	Backend
Web interface	Collects submission/reference text, calls the API, and renders the score, breakdown, and highlighted result	index.html, style.css, app.js	Frontend

6.1 Preprocessing

- Tokenizer reads the raw submission and reference text, splits it into words using a regular-expression word pattern, lower-cases and strips punctuation, and optionally removes a default stop-word list.
- Each token is wrapped in a Token object carrying its normalised word, original surface form, line number, and index, so the highlighted report can later be mapped back to the original, unnormalised text.
- A Document aggregates a file's tokens along with its source id and path, and is the unit passed into every downstream matching stage.

6.2 Exact-match detection (Rabin–Karp)

- RabinKarpMatcher slides a fixed-size window of k consecutive tokens across each reference document and computes a rolling polynomial hash for every window, indexing hash → (document, position) in a hash map.
- The same rolling hash is computed over the submission; a hash hit triggers a direct token-by-token verification to rule out hash collisions, after which the matched region is extended in both directions and recorded as a MatchSpan of type EXACT.
- Adjacent or overlapping exact spans are merged so that one long copied passage is reported once rather than as many small overlapping fragments.

6.3 Paraphrase detection (LCS dynamic programming)

- LCSAligner runs a classic O(n×m) Longest Common Subsequence dynamic-programming table between the submission and each reference document, restricted to tokens not already claimed by the exact-match stage.
- The DP table is backtracked to recover the actual aligned token positions, which are grouped into spans and emitted only when they meet a configurable minimum span length (min-span), filtering out coincidental short matches.

- These spans are recorded as MatchSpan entries of type PARAPHRASE, kept distinct from EXACT matches throughout scoring and reporting.

6.4 Scoring and reporting

- SimilarityScorer merges the exact and paraphrase spans into a single set of effective, non-overlapping spans per submission token, then computes the overall similarity percentage as covered tokens ÷ total tokens, plus a percentage contribution per reference source.
- ReportGenerator (CLI) and the equivalent logic in Server.java (API) walk the submission's token stream and wrap matched regions in highlight markers — <mark class="hl-exact"> or <mark class="hl-para"> — while leaving unmatched text untouched, producing a ready-to-render HTML report.

6.5 Connecting backend and frontend — the API bridge

Before this phase of the project, the backend was only reachable through Main.java as a command-line tool (java com.plagcheck.Main check --submission ... --reference ...), and the frontend reimplemented a simplified version of Rabin–Karp and LCS directly in JavaScript, with no connection to the Java code at all. To connect them, two new classes were added under a new com.plagcheck.api package, without modifying any existing class:

- Json.java — a small hand-written JSON parser and writer, so the server has zero external dependencies (no Gson or Jackson).
- Server.java — uses com.sun.net.httpserver.HttpServer, bundled with the JDK, to expose GET /api/health and POST /api/check. The check endpoint parses the request JSON, builds Document objects via the existing Tokenizer, calls the existing PlagiarismChecker exactly as the CLI does, and serialises the resulting ScoreResult back to JSON, including a pre-built highlighted HTML string.

On the frontend, app.js's analyze() function was rewritten to fetch() that endpoint instead of running its own matching code; the former client-side exactMatches, lcsSpans, and mergeSpans functions were removed entirely, since the real algorithms now run once, in Java, on the server. CORS headers are sent on every response so the static frontend — served separately via Python's http.server or the JDK's own jwebserver — can call the backend running on a different port.

The two processes are started independently (backend: java -cp classes com.plagcheck.api.Server 8787; frontend: jwebserver -p 5500, or the combined start.sh script) and communicate purely over localhost HTTP, keeping the backend reusable by any future client without further changes.

7. Results & Discussion

The connected system was tested end-to-end using a mix of original, verbatim-copied, and paraphrased sample documents. A sample run — a submission beginning “The quick brown fox jumps over the lazy dog...” against a reference sharing the same opening sentence — returned an overall similarity of 47.37% (9 of 19 tokens matched), with the matched sentence correctly flagged

as an EXACT span and attributed to its source document; this score was confirmed identical whether computed via the original command-line tool or via the new HTTP API, verifying that the API bridge introduced no change in the underlying algorithm's behaviour.

A second test using paragraphs on data structures and dynamic programming — one sentence copied verbatim, one paragraph reworded with synonym substitutions and reordering, and one paragraph written as entirely original content — produced a mixed EXACT and PARAPHRASE result: the verbatim sentence was flagged as EXACT, the reworded paragraph was correctly picked up by the LCS stage as PARAPHRASE even though no long contiguous substring matched, and the original paragraph was left unhighlighted, confirming that the two-stage design behaves as intended rather than only catching copy-paste plagiarism.

[Insert screenshot of the PlagiarismGuard web page here, showing the submission box, reference sources list, and the similarity score / highlighted report after clicking “Analyze plagiarism.”]

The response time for a short document (under 500 words) against two to three reference sources was well under one second on a typical laptop, since both the rolling-hash and LCS stages operate on small token arrays at this scale; very large reference corpora would require the indexing and chunking optimisations discussed in Future Scope.

8. Conclusion & Future Scope

Conclusion

This project set out to build a plagiarism checker that is both algorithmically honest — using hand-implemented Rabin–Karp hashing and LCS dynamic programming rather than a third-party library — and practically usable as a real web application. Both objectives were met: the backend correctly detects exact and paraphrased copying and reports an explainable, per-source similarity score; and, by adding a minimal, dependency-free HTTP API on top of the existing, unmodified backend classes, the previously separate frontend and backend were connected into a single working client-server system that was verified to produce identical results through both the command-line tool and the web interface.

Future Scope

Near-term:

- Add a persistent reference-document store (instead of pasting/uploading references on every run) using simple file-based or embedded-database storage on the backend.
- Add automated integration tests that start the API server and exercise it over HTTP, extending the existing SanityTest and Evaluator classes.

Medium-term:

- Introduce locality-sensitive hashing (e.g., MinHash) ahead of the exact-match stage to scale the reference corpus from a handful of documents to thousands, as discussed in the literature survey.
- Package the backend as a small standalone JAR with a simple installer/start script so non-technical users do not need to run javac/java manually.

Long-term:

- Replace the single-threaded HttpServer with a small, dependency-light concurrent server (or a lightweight framework) to support multiple simultaneous users in a classroom setting.
- Extend paraphrase detection with simple semantic-similarity techniques (e.g., word-embedding cosine similarity) alongside the existing LCS alignment, to catch synonym-heavy paraphrasing that shares few exact tokens.

9. References

8. M. O. Rabin and R. M. Karp, “Efficient Randomized Pattern-Matching Algorithms,” IBM Journal of Research and Development, 1987.
9. J. W. Hunt and M. D. McIlroy, “An Algorithm for Differential File Comparison,” Bell Laboratories Computing Science Technical Report, 1976.
10. S. Schleimer, D. S. Wilkerson, and A. Aiken, “Winnowing: Local Algorithms for Document Fingerprinting,” SIGMOD, 2003.
11. Turnitin — https://www.turnitin.com
12. Grammarly Plagiarism Checker — https://www.grammarly.com/plagiarism-checker
13. Copyleaks — https://copyleaks.com
14. Quetext — https://www.quetext.com
15. JPlag — https://github.com/jplag/JPlag
16. PlagScan — https://www.plagscan.com
17. Oracle, “com.sun.net.httpserver (Java SE 21 & JDK 21),” Java Platform, Standard Edition API Specification — https://docs.oracle.com/en/java/javase/21/docs/api/jdk.httpserver/com/sun/net/httpserver/package-summary.html